

C++ meet up – Static Reflections in C++ 26



Vasil Pashov
Senior Engineer, Man Group

Presentation to the C++ Society, Bulgaria on Thursday
19th March 2026. For educational purposes only.

Intended for C++ professionals and students only

All extracts provided in this presentation are illustrative examples
– for information only

Official working group page



Man Group



Global active investment management firm, founded in 1783

Systematic trading

One of the largest public investment management firms listed on LSE

Offices around the globe, London, Boston, Sofia among others



High performance database build in Man

Handles billions of rows with low-latency reads/writes

Open Source

Serverless

Python API

C++ 26 Static Reflections



We all did this

```
1 enum Color {RED, GREEN, BLUE};  
2  
3 const char* color_to_string(Color c) {  
4     switch(c) {  
5         case Color::RED: return "RED";  
6         case Color::GREEN: return "GREEN";  
7         case Color::BLUE: return "BLUE";  
8         default: return "Unknown";  
9     }  
10 }
```

But we can make it generic...

```
1 #include <boost/preprocessor.hpp>
2
3 #define X_DEFINE_ENUM_WITH_STRING_CONVERSIONS_TOSTRING_CASE(r, data, elem) \
4     case elem : return BOOST_PP_STRINGIZE(elem);
5
6 #define DEFINE_ENUM_WITH_STRING_CONVERSIONS(name, enumerators) \
7     enum name { \
8         BOOST_PP_SEQ_ENUM(enumerators) \
9     }; \
10 \
11     inline const char* ToString(name v) \
12     { \
13         switch (v) \
14         { \
15             BOOST_PP_SEQ_FOR_EACH( \
16                 X_DEFINE_ENUM_WITH_STRING_CONVERSIONS_TOSTRING_CASE, \
17                 name, \
18                 enumerators \
19             ) \
20             default: return "[Unknown " BOOST_PP_STRINGIZE(name) " ]"; \
21         } \
22     }
23 DEFINE_ENUM_WITH_STRING_CONVERSIONS(Color, (RED)(GREEN)(BLUE))
```

```
1 class Color(Enum):  
2     RED = 1  
3     GREEN = 2  
4     BLUE = 3  
5 def print_color(c): print(c.name)
```

```
1 enum Color { RED, GREEN, BLUE }  
2 void PrintColor(Color c) => Console.WriteLine(c.ToString());
```

The One Who Shall Not Be Named

```
1 #[derive(Debug)] enum Color { RED, GREEN, BLUE }  
2 fn print_color(c: Color) { println!("{:?}", c); }
```

Even Java...

```
1 enum Color { RED, GREEN, BLUE }  
2 static void printColor(Color c) { System.out.println(c.name()); }
```

```
1 template <typename E>
2 constexpr std::string_view enum_to_string(E value) {
3     constexpr auto enum_vals = std::define_static_array(enumulators_of(^E));
4     template for (constexpr auto e : enum_vals) {
5         if (value == [:e:]) return identifier_of(e);
6     }
7     return "<unnamed>";
8 }
9
10 enum class Color { RED, GREEN, BLUE };
11
12 void print_color(Color c) {
13     std::println("{} ", enum_to_string(c));
14 }
```

```
1 template <typename E>
2 constexpr std::string_view enum_to_string(E value) {
3     constexpr auto enum_vals = std::define_static_array(enumulators_of(^E));
4     template for (constexpr auto e : enum_vals) {
5         if (value == [:e:]) return identifier_of(e);
6     }
7     return "<unnamed>";
8 }
9
10 enum class Color { RED, GREEN, BLUE };
11
12 void print_color(Color c) {
13     std::println("{} ", enum_to_string(c));
14 }
```

```
1 template <typename E>
2 constexpr std::string_view enum_to_string(E value) {
3     constexpr auto enum_vals = std::define_static_array(enumators_of(^E));
4     template for (constexpr auto e : enum_vals) {
5         if (value == [:e:]) return identifier_of(e);
6     }
7     return "<unnamed>";
8 }
9
10 enum class Color { RED, GREEN, BLUE };
11
12 void print_color(Color c) {
13     std::println("{} ", enum_to_string(c));
14 }
```

Reflection

- The ability of a program to analyze its own structure
- Generate new code

Building blocks

- **std::meta::info**
 - Compile time handle referring a program entity
 - Opaque
 - Same type for all reflectable constructs
- **Lifting operator ^^**
 - Creates a reflection value (std::meta::info)
- **Splicing operator [::]**
 - Turns back the reflection to the underlying construct
- **Standard library methods**

Reflect and splice a type

```
1 costexpr std::meta::info refl = ^^int;  
2 [:refl:] a = 42; // int a = 42
```

Reflect and splice a function

```
1 int f(int a) {  
2     return a + 5;  
3 }  
4  
5 constexpr std::meta::info refl = ^^f;  
6 int res = [:refl:](10); // res = 15
```

Reflect a typedef

```
1 static_assert(has_identifier(^int) == false);  
2 static_assert(identifier_of(^int64_t) == "int64_t");
```

To JSON

```
1 struct MyClass {
2 public:
3     int my_public_member = 0;
4 private:
5     float my_private_member = 123;
6 };
7
8 struct Outer {
9     int my_int_var;
10    MyClass my_class_var;
11 };
12
13 template<typename T>
14 constexpr std::string to_json(const T& obj) {
15     constexpr auto ctx = std::meta::access_context::unchecked();
16     constexpr auto members = std::define_static_array(nonstatic_data_members_of(^T, ctx));
17     std::string body("{}");
18     template for(constexpr std::meta::info m : members) {
19         if constexpr(is_class_type(type_of(m))) {
20             body += std::format("{}",",", to_json(obj.[:m:]));
21         } else {
22             body += std::format("{}\": {}",",", identifier_of(m), obj.[:m:]);
23         }
24     }
25     body.back() = '}' ;
26     return std::format("{}", body);
27     // {"my_int_var": 0,{"my_public_member": 0,"my_private_member": 123}}
28 }
```

To JSON

```
1 struct MyClass {
2 public:
3     int my_public_member = 0;
4 private:
5     float my_private_member = 123;
6 };
7
8 struct Outer {
9     int my_int_var;
10    MyClass my_class_var;
11 };
12
13 template<typename T>
14 constexpr std::string to_json(const T& obj) {
15     constexpr auto ctx = std::meta::access_context::unchecked();
16     constexpr auto members = std::define_static_array(nonstatic_data_members_of(^T, ctx));
17     std::string body("{");
18     template for(constexpr std::meta::info m : members) {
19         if constexpr(is_class_type(type_of(m))) {
20             body += std::format("{}",",", to_json(obj.[:m:]));
21         } else {
22             body += std::format("{}\": {}", identifier_of(m), obj.[:m:]);
23         }
24     }
25     body.back() = '}' ;
26     return std::format("{}", body);
27     // {"my_int_var": 0, {"my_public_member": 0, "my_private_member": 123}}
28 }
```

To JSON

```
1 struct MyClass {
2 public:
3     int my_public_member = 0;
4 private:
5     float my_private_member = 123;
6 };
7
8 struct Outer {
9     int my_int_var;
10    MyClass my_class_var;
11 };
12
13 template<typename T>
14 constexpr std::string to_json(const T& obj) {
15     constexpr auto ctx = std::meta::access_context::unchecked();
16     constexpr auto members = std::define_static_array(nonstatic_data_members_of(^T, ctx));
17     std::string body("{}");
18     template for(constexpr std::meta::info m : members) {
19         if constexpr(is_class_type(type_of(m))) {
20             body += std::format("{}",",", to_json(obj.[:m:]));
21         } else {
22             body += std::format("\"{}\" : {}", identifier_of(m), obj.[:m:]);
23         }
24     }
25     body.back() = '}' ;
26     return std::format("{}", body);
27     // {"my_int_var": 0, {"my_public_member": 0, "my_private_member": 123}}
28 }
```

To JSON

```
1 struct MyClass {
2 public:
3     int my_public_member = 0;
4 private:
5     float my_private_member = 123;
6 };
7
8 struct Outer {
9     int my_int_var;
10    MyClass my_class_var;
11 };
12
13 template<typename T>
14 constexpr std::string to_json(const T& obj) {
15     constexpr auto ctx = std::meta::access_context::unchecked();
16     constexpr auto members = std::define_static_array(nonstatic_data_members_of(^T, ctx));
17     std::string body("{");
18     template for(constexpr std::meta::info m : members) {
19         if constexpr(is_class_type(type_of(m))) {
20             body += std::format("{}", to_json(obj.[:m:]));
21         } else {
22             body += std::format("{}: {},", identifier_of(m), obj.[:m:]);
23         }
24     }
25     body.back() = '}' ;
26     return std::format("{}", body);
27     // {"my_int_var": 0, {"my_public_member": 0, "my_private_member": 123}}
28 }
```

define_static_array

```
1 template<typename ElementType, ElementType... values>
2 inline constexpr ElementType arr_helper []{values...};
3
4 template<ranges::forward_range Range>
5 constexpr std::meta::info reflect_to_array(Range rng) {
6     using ElementType = ranges::range_value_t<Range>;
7     std::vector<std::meta::info> arguments{^^ElementType};
8     for(ElementType el : rng) {
9         arguments.push_back(std::meta::reflect_constant(el));
10    }
11    return substitute(^^arr_helper, arguments);
12 }
13
14 template<ranges::forward_range Range>
15 constexpr auto to_array(Range rng) {
16     using ElementType = ranges::range_value_t<Range>;
17     std::meta::info array_refl = reflect_to_array(rng);
18     return std::span{extract<const ElementType*>(array_refl), extent(type_of(array_refl))};
19 }
```

define_static_array

```
1 template<typename ElementType, ElementType... values>
2 inline constexpr ElementType arr_helper []{values...};
3
4 template<ranges::forward_range Range>
5 constexpr std::meta::info reflect_to_array(Range rng) {
6     using ElementType = ranges::range_value_t<Range>;
7     std::vector<std::meta::info> arguments{^^ElementType};
8     for(ElementType el : rng) {
9         arguments.push_back(std::meta::reflect_constant(el));
10    }
11    return substitute(^^arr_helper, arguments);
12 }
13
14 template<ranges::forward_range Range>
15 constexpr auto to_array(Range rng) {
16     using ElementType = ranges::range_value_t<Range>;
17     std::meta::info array_refl = reflect_to_array(rng);
18     return std::span{extract<const ElementType*>(array_refl), extent(type_of(array_refl))};
19 }
```

define_static_array

```
1 template<typename ElementType, ElementType... values>
2 inline constexpr ElementType arr_helper []{values...};
3
4 template<ranges::forward_range Range>
5 constexpr std::meta::info reflect_to_array(Range rng) {
6     using ElementType = ranges::range_value_t<Range>;
7     std::vector<std::meta::info> arguments{^^ElementType};
8     for(ElementType el : rng) {
9         arguments.push_back(std::meta::reflect_constant(el));
10    }
11    return substitute(^^arr_helper, arguments);
12 }
13
14 template<ranges::forward_range Range>
15 constexpr auto to_array(Range rng) {
16     using ElementType = ranges::range_value_t<Range>;
17     std::meta::info array_refl = reflect_to_array(rng);
18     return std::span{extract<const ElementType*>(array_refl), extent(type_of(array_refl))};
19 }
```

define_static_array

```
1 template<typename ElementType, ElementType... values>
2 inline constexpr ElementType arr_helper []{values...};
3
4 template<ranges::forward_range Range>
5 constexpr std::meta::info reflect_to_array(Range rng) {
6     using ElementType = ranges::range_value_t<Range>;
7     std::vector<std::meta::info> arguments{^^ElementType};
8     for(ElementType el : rng) {
9         arguments.push_back(std::meta::reflect_constant(el));
10    }
11    return substitute(^^arr_helper, arguments);
12 }
13
14 template<ranges::forward_range Range>
15 constexpr auto to_array(Range rng) {
16     using ElementType = ranges::range_value_t<Range>;
17     std::meta::info array_refl = reflect_to_array(rng);
18     return std::span{extract<const ElementType*>(array_refl), extent(type_of(array_refl))};
19 }
```

To struct of arrays

```

1 struct color {float r, g, b;};
2 struct position{float x, y, z;};
3 struct vertex{color color; position position;};
4
5 template<typename S>
6 struct ToStructOfArrays {
7     struct StructOfArrays;
8     constexpr {
9         constexpr auto ctx = std::meta::access_context::current();
10        constexpr auto members = std::define_static_array(nonstatic_data_members_of(^S, ctx));
11        std::vector<std::meta::info> new_members;
12        template for(constexpr auto member: members) {
13            std::meta::info type = substitute(^std::vector, {type_of(member)});
14            new_members.push_back(data_member_spec(type, {.name = identifier_of(member)}));
15        }
16        define_aggregate(^^StructOfArrays, new_members);
17    }
18
19 };
20
21 int main() {
22     typename ToStructOfArrays<vertex>::StructOfArrays my_soa;
23     constexpr auto ctx = std::meta::access_context::current();
24     constexpr auto mems = std::define_static_array(nonstatic_data_members_of(type_of(^my_soa), ctx));
25     template for(constexpr auto member: mems) {
26         std::println("{}: {}", display_string_of(type_of(member)), display_string_of(member));
27     }
28 }

```

To struct of arrays

```

1 struct color {float r, g, b;};
2 struct position{float x, y, z;};
3 struct vertex{color color; position position;};
4
5 template<typename S>
6 struct ToStructOfArrays {
7     struct StructOfArrays;
8     constexpr {
9         constexpr auto ctx = std::meta::access_context::current();
10        constexpr auto members = std::define_static_array(nonstatic_data_members_of(^S, ctx));
11        std::vector<std::meta::info> new_members;
12        template for(constexpr auto member: members) {
13            std::meta::info type = substitute(^std::vector, {type_of(member)});
14            new_members.push_back(data_member_spec(type, {.name = identifier_of(member)}));
15        }
16        define_aggregate(^^StructOfArrays, new_members);
17    }
18
19 };
20
21 int main() {
22     typename ToStructOfArrays<vertex>::StructOfArrays my_soa;
23     constexpr auto ctx = std::meta::access_context::current();
24     constexpr auto mems = std::define_static_array(nonstatic_data_members_of(type_of(^my_soa), ctx));
25     template for(constexpr auto member: mems) {
26         std::println("{}: {}", display_string_of(type_of(member)), display_string_of(member));
27     }
28 }

```

To struct of arrays

```

1 struct color {float r, g, b;};
2 struct position{float x, y, z;};
3 struct vertex{color color; position position;};
4
5 template<typename S>
6 struct ToStructOfArrays {
7     struct StructOfArrays;
8     constexpr {
9         constexpr auto ctx = std::meta::access_context::current();
10        constexpr auto members = std::define_static_array(nonstatic_data_members_of(^S, ctx));
11        std::vector<std::meta::info> new_members;
12        template for(constexpr auto member: members) {
13            std::meta::info type = substitute(^std::vector, {type_of(member)});
14            new_members.push_back(data_member_spec(type, {.name = identifier_of(member)}));
15        }
16        define_aggregate(^^StructOfArrays, new_members);
17    }
18
19 };
20
21 int main() {
22     typename ToStructOfArrays<vertex>::StructOfArrays my_soa;
23     constexpr auto ctx = std::meta::access_context::current();
24     constexpr auto mems = std::define_static_array(nonstatic_data_members_of(type_of(^my_soa), ctx));
25     template for(constexpr auto member: mems) {
26         std::println("{}: {}", display_string_of(type_of(member)), display_string_of(member));
27     }
28 }

```

To struct of arrays

```
1 struct color {float r, g, b;};
2 struct position{float x, y, z;};
3 struct vertex{color color; position position;};
4
5 template<typename S>
6 struct ToStructOfArrays {
7     struct StructOfArrays;
8     constexpr {
9         constexpr auto ctx = std::meta::access_context::current();
10        constexpr auto members = std::define_static_array(nonstatic_data_members_of(^S, ctx));
11        std::vector<std::meta::info> new_members;
12        template for(constexpr auto member: members) {
13            std::meta::info type = substitute(^std::vector, {type_of(member)});
14            new_members.push_back(data_member_spec(type, {.name = identifier_of(member)}));
15        }
16        define_aggregate(^StructOfArrays, new_members);
17    }
18 }
19 };
20
21 int main() {
22     typename ToStructOfArrays<vertex>::StructOfArrays my_soa;
23     constexpr auto ctx = std::meta::access_context::current();
24     constexpr auto mems = std::define_static_array(nonstatic_data_members_of(type_of(^my_soa), ctx));
25     template for(constexpr auto member: mems) {
26         std::println("{}: {}", display_string_of(type_of(member)), display_string_of(member));
27     }
28 }
```

To struct of arrays

```

1 struct color {float r, g, b;};
2 struct position{float x, y, z;};
3 struct vertex{color color; position position;};
4
5 template<typename S>
6 struct ToStructOfArrays {
7     struct StructOfArrays;
8     constexpr {
9         constexpr auto ctx = std::meta::access_context::current();
10        constexpr auto members = std::define_static_array(nonstatic_data_members_of(^S, ctx));
11        std::vector<std::meta::info> new_members;
12        template for(constexpr auto member: members) {
13            std::meta::info type = substitute(^std::vector, {type_of(member)});
14            new_members.push_back(data_member_spec(type, {.name = identifier_of(member)}));
15        }
16        define_aggregate(^^StructOfArrays, new_members);
17    }
18 }
19 };
20
21 int main() {
22     typename ToStructOfArrays<vertex>::StructOfArrays my_soa;
23     constexpr auto ctx = std::meta::access_context::current();
24     constexpr auto mems = std::define_static_array(nonstatic_data_members_of(type_of(^my_soa), ctx));
25     template for(constexpr auto member: mems) {
26         std::println("{}: {}", display_string_of(type_of(member)), display_string_of(member));
27     }
28 }

```

To struct of arrays

```

1 struct color {float r, g, b;};
2 struct position{float x, y, z;};
3 struct vertex{color color; position position;};
4
5 template<typename S>
6 struct ToStructOfArrays {
7     struct StructOfArrays;
8     constexpr {
9         constexpr auto ctx = std::meta::access_context::current();
10        constexpr auto members = std::define_static_array(nonstatic_data_members_of(^S, ctx));
11        std::vector<std::meta::info> new_members;
12        template for(constexpr auto member: members) {
13            std::meta::info type = substitute(^std::vector, {type_of(member)});
14            new_members.push_back(data_member_spec(type, {.name = identifier_of(member)}));
15        }
16        define_aggregate(^StructOfArrays, new_members);
17    }
18
19 };
20
21 int main() {
22     typename ToStructOfArrays<vertex>::StructOfArrays my_soa;
23     constexpr auto ctx = std::meta::access_context::current();
24     constexpr auto mems = std::define_static_array(nonstatic_data_members_of(type_of(^my_soa), ctx));
25     template for(constexpr auto member: mems) {
26         std::println("{}: {}", display_string_of(type_of(member)), display_string_of(member));
27     }
28 }

```

To struct of arrays

```
1 struct color {float r, g, b;};
2 struct position{float x, y, z;};
3 struct vertex{color color; position position;};
4
5 template<typename S>
6 struct ToStructOfArrays {
7     struct StructOfArrays;
8     constexpr {
9         constexpr auto ctx = std::meta::access_context::current();
10        constexpr auto members = std::define_static_array(nonstatic_data_members_of(^S, ctx));
11        std::vector<std::meta::info> new_members;
12        template for(constexpr auto member: members) {
13            std::meta::info type = substitute(^std::vector, {type_of(member)});
14            new_members.push_back(data_member_spec(type, {.name = identifier_of(member)}));
15        }
16        define_aggregate(^StructOfArrays, new_members);
17    }
18
19 };
20
21 int main() {
22     typename ToStructOfArrays<vertex>::StructOfArrays my_soa;
23     constexpr auto ctx = std::meta::access_context::current();
24     constexpr auto mems = std::define_static_array(nonstatic_data_members_of(type_of(^my_soa), ctx));
25     template for(constexpr auto member: mems) {
26         std::println("{}: {}", display_string_of(type_of(member)), display_string_of(member));
27     }
28 }
```

Bonus - Annotations

```

1 struct Opts {
2     template<size_t N>
3     struct Help(char value[N]);
4     struct Required{bool value;};
5 };
6
7 struct Args {
8     [[=Opts::Help("Student name")]]
9     [[=Opts::Required{true}]]
10    std::string name;
11
12    [[=Opts::Help("Student age")]]
13    [[=Opts::Required{false}]]
14    int age;
15 };
16
17 template<typename Args>
18 Args parse(const char** argc, size_t argv) {
19     Args result;
20     std::span<const char*> input{argc, argv};
21     constexpr auto ctx = std::meta::access_context::unchecked();
22     constexpr auto members = std::define_static_array(nonstatic_data_members_of(^Args, ctx));
23     template for(constexpr auto member : members) {
24         constexpr static auto name = identifier_of(member);
25         constexpr auto annotations = std::define_static_array(annotations_of(member));
26         constexpr bool required = (extract<Opts::Required>(
27             *std::ranges::find_if(
28                 annotations,
29                 [](auto arg_annotation) {return type_of(arg_annotation) == ^Opts::Required;})
30             ).value;
31         auto argument = std::ranges::find_if(
32             input,
33             [](std::string_view a) {return a.starts_with("--") && a.substr(2) == name;})
34         );
35         if(argument == input.end() && required) {
36             throw std::runtime_error(std::format("{}\n was required but is not provided", name));
37         } else if(argument != input.end()) {
38             std::string_view value = *(argument + 1);
39             if constexpr(type_of(member) == dealias(^std::string)) {
40                 result[member:] = value;
41             } else {
42                 result[member:] = std::stoi(*(argument+1));
43             }
44         }
45     }
46 }
47
48 return result;
49 }
50
51 int main(int argc, char** argv) {
52     std::array args{"--name", "MyUser", "--age", "30"};
53     Args arg = parse<Args>(args.data(), args.size());
54     std::println("{} {}", arg.name, arg.age);
55 }

```

Important Information

This presentation is provided for educational purposes only.

This information is communicated and/or distributed by the relevant Man entity identified below (collectively the "Company") subject to the following conditions and restriction in their respective jurisdictions.

Opinions expressed are those of the author and may not be shared by all personnel of Man Group plc ('Man'). These opinions are subject to change without notice, are for information purposes only and do not constitute an offer or invitation to make an investment in any financial instrument or in any product to which the Company and/or its affiliates provides investment advisory or any other financial services. Any organisations, financial instrument or products described in this material are mentioned for reference purposes only which should not be considered a recommendation for their purchase or sale. Neither the Company nor the authors shall be liable to any person for any action taken on the basis of the information provided. Some statements contained in this material concerning goals, strategies, outlook or other non-historical matters may be forward-looking statements and are based on current indicators and expectations. These forward-looking statements speak only as of the date on which they are made, and the Company undertakes no obligation to update or revise any forward-looking statements. These forward-looking statements are subject to risks and uncertainties that may cause actual results to differ materially from those contained in the statements. The Company and/or its affiliates may or may not have a position in any financial instrument mentioned and may or may not be actively trading in any such securities. Unless stated otherwise all information is provided by the Company. Past performance is not indicative of future results.

Unless stated otherwise this information is communicated by the relevant entity listed below.

Australia: To the extent this material is distributed in Australia it is communicated by Man Investments Australia Limited ABN 47 002 747 480 AFSL 240581, which is regulated by the Australian Securities & Investments Commission ('ASIC'). This information has been prepared without taking into account anyone's objectives, financial situation or needs. **Austria/Germany/Liechtenstein:** To the extent this material is distributed in Austria, Germany and/or Liechtenstein it is communicated by Man (Europe) AG, which is authorised and regulated by the Liechtenstein Financial Market Authority (FMA). Man (Europe) AG is registered in the Principality of Liechtenstein no. FL-0002.420.371-2. Man (Europe) AG is an associated participant in the investor compensation scheme, which is operated by the Deposit Guarantee and Investor Compensation Foundation PCC (FL-0002.039.614-1) and corresponds with EU law. Further information is available on the Foundation's website under www.eas-liechtenstein.li.

European Economic Area: Unless indicated otherwise this material is communicated in the European Economic Area by Man Asset Management (Ireland) Limited ('MAMIL') which is registered in Ireland under company number 250493 and has its registered office at 70 Sir John Rogerson's Quay, Grand Canal Dock, Dublin 2, Ireland. MAMIL is authorised and regulated by the Central Bank of Ireland under number C22513. **Hong Kong SAR:** To the extent this material is distributed in Hong Kong SAR, this material is communicated by Man Investments (Hong Kong) Limited and has not been reviewed by the Securities and Futures Commission in Hong Kong. **Japan:** To the extent this material is distributed in Japan it is communicated by Man Group Japan Limited, Financial Instruments Business Operator, Director of Kanto Local Finance Bureau (Financial instruments firms) No. 624 for the purpose of providing information on investment strategies, investment services, etc. provided by Man Group, and is not a disclosure document based on laws and regulations. This material can only be communicated only to professional investors (i.e. specific investors or institutional investors as defined under Financial Instruments Exchange Law) who may have sufficient knowledge and experience of related risks.

Switzerland: To the extent this material is made available in Switzerland the communicating entity is Man Investments AG, Huobstrasse 3, 8808 Pfäffikon SZ, Switzerland, which is regulated by the Swiss Financial Market Supervisory Authority ('FINMA').

United Kingdom: Unless indicated otherwise this material is communicated in the United Kingdom by Man Solutions Limited ('MSL') which is a private limited company registered in England and Wales under number 3385362. MSL is authorised and regulated by the UK Financial Conduct Authority (the 'FCA') under number 185637 and has its registered office at Riverbank House, 2 Swan Lane, London, EC4R 3AD, United Kingdom.

United States: To the extent this material is distributed in the United States, it is communicated and distributed by Man Investments, Inc. ('Man Investments'). Man Investments is registered as a broker-dealer with the SEC and is a member of the Financial Industry Regulatory Authority ('FINRA'). Man Investments is also a member of the Securities Investor Protection Corporation ('SIPC'). Man Investments is a wholly owned subsidiary of Man Group plc. The registration and memberships described above in no way imply a certain level of skill or expertise or that the SEC, FINRA or the SIPC have endorsed Man Investments. Man Investments Inc, 1345 Avenue of the Americas, 21st Floor, New York, NY 10105..

This material is proprietary information and may not be reproduced or otherwise disseminated in whole or in part without prior written consent. Any data services and information available from public sources used in the creation of this material are believed to be reliable. However accuracy is not warranted or guaranteed. © Man 2026